



INSTITUTO POLITECNICO NACIONAL



Escuela Superior de Cómputo
(ESCOM)

Ingeniería en sistemas computacionales

Materia:

Bases de Datos

-Gabriel Hurtado Avilés-

Practica 1

Modelo Entidad Relación

Evidencias (1,2)

3CV4

Cortés Rios Hazel Alexis

INTRODUCCIÓN

Ejercicio 1

Sistema control de versiones

Un sistema de control de versiones es una herramienta que permite registrar, organizar y administrar los cambios que se realizan en los archivos de un proyecto a lo largo del tiempo. Su principal objetivo es mantener un historial de las modificaciones realizadas, de manera que sea posible conocer qué cambios se hicieron, quién los realizó y, en caso de ser necesario, regresar a una versión anterior del proyecto.

Este tipo de sistemas son especialmente importantes cuando varias personas trabajan en un mismo proyecto. Sin un sistema de control de versiones, cada integrante podría modificar los mismos archivos y posteriormente sería complicado determinar qué cambios deben conservarse. Incluso podrían sobrescribirse accidentalmente los cambios realizados por otra persona

Diferencia entre Git y GITHUB

Git es un sistema de control de versiones distribuido. Es un programa que se instala en una computadora y permite registrar los cambios de un proyecto, crear ramas, realizar commits, comparar versiones y fusionar modificaciones. Git puede utilizarse sin necesidad de estar conectado a Internet.

Por otro lado, GitHub es una plataforma en línea que permite almacenar repositorios Git y colaborar con otras personas. Además de almacenar el código, proporciona herramientas para trabajar en equipo, como Pull Requests, revisión de código, seguimiento de problemas y administración de proyectos.

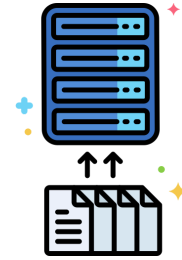
Una forma sencilla de entender la diferencia es pensar que Git es la herramienta que controla las versiones del proyecto, mientras que GitHub es una plataforma que permite alojar esos proyectos y facilitar la colaboración entre personas.

Conceptos principales

Repositorio

Un repositorio es el espacio donde se almacena un proyecto y su historial de cambios. Puede contener código fuente, documentos, imágenes y otros archivos relacionados con el proyecto.

Por ejemplo, si cuatro estudiantes desarrollan una aplicación para administrar alumnos, pueden crear un repositorio llamado sistema-escolar. Dentro de él estarán los archivos del programa y el historial de las modificaciones realizadas.



Confirmación o Commit

Un commit es una confirmación de los cambios realizados en un proyecto. Funciona como un punto guardado dentro del historial, permitiendo saber qué modificaciones se hicieron en determinado momento.

Por ejemplo, después de terminar el módulo de inicio de sesión, un estudiante puede realizar un commit con un mensaje como:

Agregar sistema de inicio de sesión

Rama o Branch

Una rama (branch) es una línea de desarrollo independiente dentro de un repositorio. Permite trabajar en una nueva característica o modificación sin alterar directamente la versión principal del proyecto.

Por ejemplo, si el proyecto principal se encuentra en la rama main, un estudiante puede crear una rama llamada login para desarrollar el sistema de inicio de sesión. Mientras trabaja en ella, la rama principal permanece sin modificaciones causadas por ese trabajo.

Fusión o Merge

Un merge es el proceso mediante el cual se combinan los cambios realizados en una rama con otra rama.

Por ejemplo, si un estudiante terminó correctamente la funcionalidad de inicio de sesión en la rama login, sus cambios pueden fusionarse con main para incorporarlos al proyecto principal.

Conflicto de fusión

Un conflicto de fusión (merge conflict) ocurre cuando Git no puede combinar automáticamente dos modificaciones porque existen cambios incompatibles en la misma parte de un archivo.

Por ejemplo, si dos estudiantes modifican exactamente la misma línea de un archivo: uno cambia precio = 100 por precio = 120, mientras que el otro cambia esa misma línea por precio = 150, Git no puede decidir cuál de los dos valores debe conservarse. En este caso, los integrantes deben revisar el conflicto y decidir manualmente cuál modificación es correcta.

Pull Request

Un Pull Request (PR) es una solicitud para incorporar los cambios de una rama a otra, normalmente hacia la rama principal. Además de solicitar la fusión, permite que otros integrantes del equipo revisen el código antes de incorporarlo al proyecto.

Por ejemplo, después de terminar una nueva función en la rama login, un estudiante puede crear un Pull Request para solicitar que sus cambios sean revisados y posteriormente fusionados con main.

Archivo .gitignore

El archivo .gitignore sirve para indicarle a Git qué archivos o carpetas no deben incluirse en el control de versiones.

Esto es útil para evitar subir archivos que no son necesarios para el proyecto, como archivos temporales, configuraciones locales, archivos generados automáticamente o información que no debe compartirse.

Por ejemplo, en un proyecto de programación se pueden ignorar carpetas generadas por el compilador o archivos de configuración específicos de la computadora de cada desarrollador.

Archivo README

El archivo README es un documento que proporciona información sobre el proyecto. Generalmente explica qué hace el programa, cómo instalarlo, cómo utilizarlo y otros datos importantes para las personas que quieran conocer o ejecutar el proyecto.

Por ejemplo, un repositorio de un sistema escolar puede incluir un README que explique que el programa permite registrar alumnos, consultar materias y generar reportes, además de indicar los pasos necesarios para instalarlo.

Flujo de trabajo basado en ramas y revisión entre pares

Un flujo de trabajo basado en ramas consiste en que los integrantes de un equipo no realizan todas sus modificaciones directamente sobre la rama principal. En lugar de ello, cada integrante crea una rama independiente para trabajar en una característica, corrección o mejora específica.

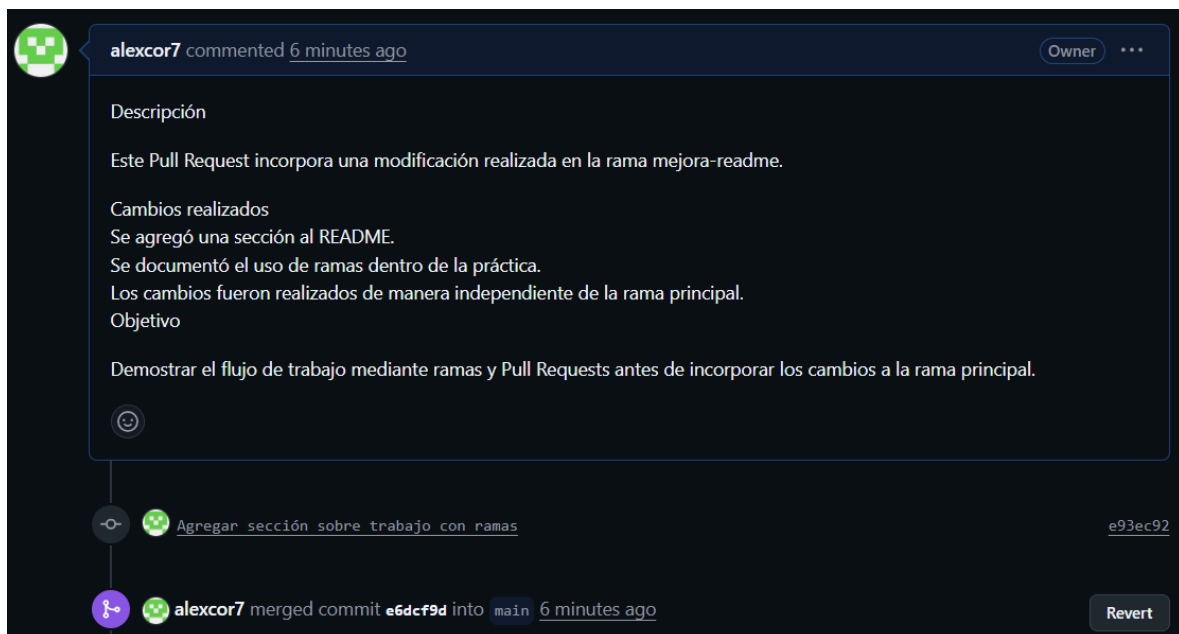
Por ejemplo, un equipo puede utilizar la siguiente estructura:

- main: contiene la versión estable del proyecto.
- login: desarrollo del inicio de sesión.
- registro: desarrollo del registro de usuarios.
- reportes: desarrollo de los reportes.

Cada integrante trabaja en su propia rama y realiza commits conforme avanza. Cuando termina una tarea, puede crear un Pull Request para solicitar que sus cambios sean incorporados a main.

El código se revisa entre pares antes de fusionarse porque otra persona puede detectar errores que el autor no observó. La revisión también permite comprobar que el código sea comprensible, siga las reglas establecidas por el equipo y no introduzca problemas en otras partes del programa.

Además, la revisión entre compañeros ayuda a mantener una mejor calidad del proyecto. En lugar de que una sola persona sea responsable de detectar todos los errores, varios integrantes pueden analizar los cambios antes de incorporarlos a la versión principal.



alexcor7 commented 6 minutes ago Owner ...

Descripción

Este Pull Request incorpora una modificación realizada en la rama mejora-readme.

Cambios realizados

- Se agregó una sección al README.
- Se documentó el uso de ramas dentro de la práctica.
- Los cambios fueron realizados de manera independiente de la rama principal.

Objetivo

Demostrar el flujo de trabajo mediante ramas y Pull Requests antes de incorporar los cambios a la rama principal.

 **Agregar sección sobre trabajo con ramas** e93ec92

 **alexcor7** merged commit **e6dcf9d** into **main** 6 minutes ago Revert

```
PS C:\Users\Changos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical> git log --oneline --graph --all
* e6dcf9d (HEAD -> main, origin/main) Merge pull request #1 from alexcor7/mejora-readme
|
| * e93ec92 (origin/mejora-readme, mejora-readme) Agregar sección sobre trabajo con ramas
|/
* 6d484c0 Documentar comandos utilizados en la práctica
* 03a9cdb Ampliar documentación de comandos
* cd05158 Agregar documentación de comandos Git
* 3dba7fe Agregar archivo gitignore para archivos innecesarios
* d3098c6 Agregar README inicial de la práctica
```

Ejercicio 2. El sistema gestor en un contenedor: Docker

Contenedor

Un contenedor es una unidad aislada de software que contiene una aplicación junto con los archivos, dependencias y configuraciones necesarias para ejecutarla. Los contenedores utilizan el sistema operativo del equipo anfitrión, pero mantienen aislados sus procesos y recursos de otros programas y contenedores.

Una diferencia importante respecto a una máquina virtual (VM) es que una máquina virtual necesita ejecutar un sistema operativo completo dentro de ella mediante un hipervisor. Por esta razón, normalmente requiere más espacio de almacenamiento y memoria, y su arranque suele ser más lento. Un contenedor, en cambio, comparte el núcleo del sistema operativo anfitrión y únicamente ejecuta los componentes necesarios para la aplicación, por lo que generalmente es más ligero y arranca en pocos segundos o incluso menos.

Definiciones

Imagen: es una plantilla inmutable que contiene los archivos, dependencias y configuración necesarios para crear un contenedor. Por ejemplo, una imagen de PostgreSQL contiene los componentes necesarios para ejecutar el sistema gestor de bases de datos.

Contenedor: es una instancia en ejecución de una imagen. A partir de una imagen de PostgreSQL se puede crear un contenedor que ejecute el servidor de bases de datos.

Volumen: es un mecanismo de Docker para almacenar datos de forma independiente del ciclo de vida del contenedor. Esto permite que los datos de una aplicación, como las bases de datos de PostgreSQL, permanezcan aunque el contenedor sea eliminado.

Puerto publicado: es la forma en que Docker hace accesible desde el sistema anfitrión un puerto que está siendo utilizado dentro del contenedor. Por ejemplo, PostgreSQL utiliza normalmente el puerto 5432, por lo que se puede publicar como 5432:5432, donde el primer número corresponde al puerto del equipo anfitrión y el segundo al puerto dentro del contenedor.

En una base de datos, el volumen es fundamental porque los datos deben sobrevivir al ciclo de vida del contenedor. El contenedor puede detenerse, reiniciarse o incluso eliminarse, pero la información almacenada en la base de datos debe permanecer.

Si PostgreSQL se ejecuta en un contenedor sin utilizar un volumen para almacenar sus datos, la información queda asociada a la capa de almacenamiento del propio contenedor. Mientras el contenedor solamente se detenga, los datos normalmente seguirán disponibles cuando se vuelva a iniciar. Sin embargo, si el contenedor se elimina, esa capa de almacenamiento también se elimina y los datos que estaban dentro de ella se pierden.

```
ca Símbolo del sistema - docker exec -it BD-practica1 psql -U postgres

Microsoft Windows [Versión 10.0.19045.6216]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Changos>docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                    NAMES
e4811f86d84e   postgres:17 "docker-entrypoint.s..." About an hour ago Up 56 minutes   5432/tcp              BD-practica
54fd5d64ee59   postgres:17 "docker-entrypoint.s..." 11 days ago   Up About an hour   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   postgres173

C:\Users\Changos>docker exec -it BD-practica1 psql -U postgres
psql (17.11 (Debian 17.11-1.pgdg13+2))
Type "help" for help.

postgres=# CREATE DATABASE practical1;
CREATE DATABASE
postgres=# \l

                                List of databases
  Name | Owner  | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 postgres | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | 
 practical1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | 
 template0 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
 template1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
                                     postgres=CtC/postgres
(4 rows)

postgres=# \c practical1
You are now connected to database "practical1" as user "postgres".
practical1=#
```

```
C:\Users\Changos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical\entorno>docker compose down
[+] down 2/2
0 Container BD-practica1 Removed 1.6s Network entorno_default Removed

C:\Users\Changos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical\entorno>docker volume ls
DRIVER VOLUME NAME

C:\Users\Changos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical\entorno>docker compose up -d
[+] up 2/2hngos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical\entorno>docker compose up -d
0.3s Network entorno_default Created 4.3s Container BD-practica1 Started

C:\Users\Changos\OneDrive\Desktop\ESCOM\Base de Datos\BD-Practical\entorno>docker exec -it BD-practica1 psql -U postgres -c "\l"

                                List of databases
  Name | Owner  | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
 postgres | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | 
 practical1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | 
 template0 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
 template1 | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |         |          | =c/postgres +
                                     postgres=CtC/postgres
(4 rows)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug BD-practica1
Learn more at https://docs.docker.com/go/debug-cli/
```

BIBLIOGRAFÍA

- Git. (s. f.). *Git documentation*. <https://git-scm.com/docs>
- Microsoft. (27 de octubre de 2025). *¿Qué es el control de versiones?* Microsoft Learn. <https://learn.microsoft.com/es-es/devops/develop/git/what-is-version-control>
- PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>